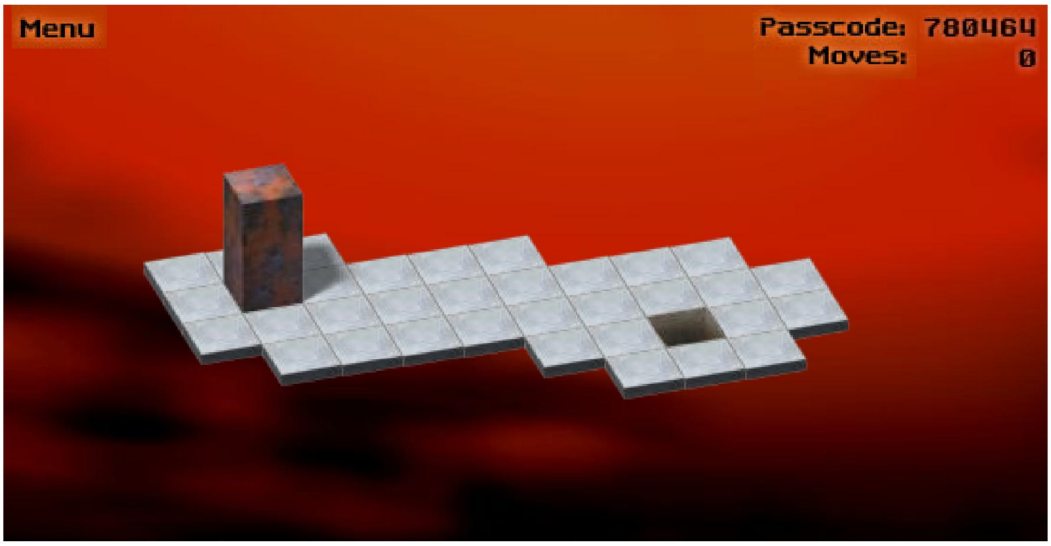
Project 1

Bloxorz Solver

1 Introduction

Bloxorz[1] is a popular block-rolling puzzle game in which the player controls a single $1 \times 1 \times 2$ rectangular block resting on a floor made up of square tiles. The block is rolled around the board, and the objective of the game is to make the block fall standing upright into a square goal hole. The challenge lies in the fact that the block is not a single point: depending on whether it is *standing* or *lying*, it occupies different tiles and reacts differently to the floor. Rolling the block off the edge of the board results in a loss.



Hình 1: A screenshot of a typical Bloxorz gameplay.

This game is particularly suitable for practicing search algorithms such as BFS, DFS, UCS, and A*. It is easily accessible: students can grasp the rules within minutes and relate them to a search problem. The game is also *fully observable* and *deterministic*: the board is fully visible and each roll leads to exactly one successor state, with no random or adversarial elements. Most interestingly, the **orientation** of the block is an essential part of the state, not just its coordinate, making the problem an excellent vehicle for designing admissible heuristics and analyzing informed methods.

[1] Bloxorz - Coolmath Games: https://www.coolmathgames.com/0-bloxorz

2 Gameplay

The following rules are adapted from the original Coolmath Games version. You can try the game via this link.

2.1 Construction and layout

A stage is a finite grid of square cells. Each cell is one of the following types:

- **Floor tile**: a normal, solid tile the block may rest on.
- **Empty cell (void)**: there is no tile; if any part of the block ends up over a void, the block falls and the move is illegal.
- **Goal hole**: the single target cell; the stage is solved only when the block falls into it.

The block starts standing upright on a designated starting tile.

2.2 Block orientation

Because the block is $1 \times 1 \times 2$, it always exists exactly in one of three orientations, which is a core part of the game state:

- **Standing**: the block occupies a single cell (r, c).
- **Lying horizontally**: the block occupies two horizontally adjacent cells, (r, c) and $(r, c+1)$.
- **Lying vertically**: the block occupies two vertically adjacent cells, (r, c) and $(r+1, c)$.

2.3 Moves

On each turn, the player will roll the block in one of four directions: **Up**, **Down**, **Left**, or **Right**. The result depends on the current orientation: a standing block tips over and ends up lying across the two cells in the rolling direction; a block lying along the rolling direction stands up onto a single cell; and a block lying across the rolling direction rolls sideways and remains lying, shifted by one cell.

A move is **illegal** (not allowed) if, after rolling, any cell the block would occupy is a void cell or lies outside the board. In other words, the block must always be fully supported by floor tiles.

2.4 Victory

A stage is won when the block falls into the goal hole. This happens only when the block is **standing upright** on the goal cell. A block that merely lies across the goal cell does **not** win.

2.5 Advanced tiles

Each stage may include the following special tiles. These tiles introduce additional gameplay mechanics by altering how the block interacts with the board.

- **Fragile tiles**: A fragile (orange) tile can support the block only when its weight is distributed (the block is lying). If the block stands on a fragile tile, the tile breaks and the block falls. The block may safely lie across fragile tiles.
- **Bridges**: A bridge is a tile whose passability is controlled by a switch. A *closed* bridge behaves like empty space, while an *open* bridge behaves like a normal floor tile. Each switch is linked to one or more bridge tiles.

The game features **three** types of switches:

- **Soft switch (octagon / circle)**: Activated when *any* part of the block presses it, regardless of orientation (standing or lying).
- **Heavy switch (X-shaped)**: Activated only when the block is **standing** on it; lying on it does not trigger the switch.
- **Split switch (bracket-shaped "( )")**: Activated only when the block is **standing** on it. It teleports the block to predefined positions and splits it into two $1 \times 1 \times 1$ cubes.

When activated, a switch affects its linked bridges in one of two ways: **toggle** (each activation switches the bridges between open and closed states) or **permanent** (the bridges are set to a fixed state after activation).

Block splitting. After being split into two unit cubes, they are controlled **one at a time** (e.g., switching control with the spacebar) and merge back into a standard $1 \times 1 \times 2$ block when positioned on adjacent tiles. A single cube can activate soft switches but **cannot** activate heavy switches or complete the stage.

3 Requirements

Before we get started, here are some important things to consider:

- The source code must be written in **Python**.
- You may use supporting libraries (e.g., for GUI or data structures), but you must implement the search algorithms **yourself**.
- There is **no restriction** on how you organize your code, but it should be professionally and clearly structured.

3.1 Reimplement the Game (20 pts)

Implement a basic and playable version of the Bloxorz game. The program **MUST HAVE** a GUI (a web or desktop application is acceptable). This task is split into two equally weighted parts:

- **Core mechanics (10 pts)**: the standard gameplay – a board of floor tiles, void cells, and a goal hole; a $1 \times 1 \times 2$ block with the three orientations; the four rolling moves with correct orientation transitions; illegal-move detection (falling off the board or over voids); and win detection (the block standing in the goal hole).
- **Advanced tiles (10 pts)**: **fragile tiles**, **bridges**, and all **three switch types** (soft, heavy, split, with block splitting/rejoining and both bridge behaviours) as described in Section 2.5. Your solvers must also handle these tiles correctly.

F.Y.I:

- Pay close attention to the orientation transition table: each (orientation, direction) pair maps deterministically to a new (orientation, position). Implementing and unit-testing this table early will save a lot of debugging.
- With switches and bridges, the **state must include the open/closed status of every bridge**, not just the block's position and orientation: states with the same block placement but different bridge configurations are *different* states.
- You may design your own stages or recreate classic levels. Provide a simple text/JSON format for loading stages so the same stage can be reused across all solvers.

The GUI must also display the basic controls (e.g., New Game, Restart, and buttons for invoking the following solvers).

3.2 BFS Solver (10 pts)

- Implement a Bloxorz solver using **Breadth-First Search (BFS)**.
- The solver must record the Search Time, Memory Usage, Expanded Nodes, and Solution Length for later analysis in the experiments section. The same results should be recorded for each of the following solvers.

3.3 DFS Solver (10 pts)

- Implement a Bloxorz solver using **Depth-First Search (DFS)**. You may use **Iterative Deepening Search (IDS)** instead if you prefer; state this clearly in your report.
- Handle repeated states to avoid getting stuck in cycles (e.g., a visited set or hashing).

3.4 UCS Solver (10 pts)

- Implement a Bloxorz solver using **Uniform-Cost Search (UCS)**.
- *In the report*, clearly **define and justify your cost function**. For example, each roll may cost 1 (minimizing the number of moves), or different moves may have different costs (e.g., rolling onto a fragile tile is "expensive"because it is risky, while toggling a switch is "cheap"), as long as you give a reasonable explanation.
- Note that if every move has cost 1, UCS reduces to BFS; designing a non-uniform cost function (and explaining the intuition behind it) is what makes this task meaningful.

3.5 A* Solver (10 pts)

- Implement a Bloxorz solver using **A* Search** with an **appropriate heuristic** that estimates how close a given state is to the goal.
- Some non-mandatory heuristic ideas:
  - The Manhattan distance from the block to the goal cell, divided by 2 (since a single roll can move the block by at most two cells).
  - An orientation-aware term: if the block is not standing, it needs at least one extra roll to stand up on the goal, so a small additional penalty can be added.

  - A relaxed "point-object"distance: ignore the block's orientation and the void/fragile constraints, treat the block as a single point moving on a 4-connected grid, and take the shortest-path distance to the goal as a lower bound.

- *In the report*:
  - Explain how you designed your heuristic.
  - Discuss whether it is *admissible* (never overestimates the true remaining cost) and/or *consistent*, with clear arguments. Be careful: your heuristic must remain admissible *with respect to your chosen cost function.*

3.6 Report (30 pts)

Submit a well-formatted PDF report including:

- **Project Planning and Task Distribution**:
  - Document the responsibilities of each team member, including which tasks were assigned and each member's completion percentage.
  - Example: if the group score is 9.0 and Student A has a completion percentage of 90%, then A's individual score is $9.0 \times 90\% = 8.1$.
  - It is important to evaluate contributions **fairly** and **accurately**.
- **Algorithm Description (10 pts)**:
  - Provide detailed explanations of each search algorithm used (BFS, DFS/IDS, UCS, A*).
  - Describe how you formalize the search problem: state representation (including the orientation and bridge configuration), actions, transition model, goal test, cost function, heuristics, and how you avoid revisiting states.
- **Experiments (20 pts)**:
  - Evaluate search performance on multiple test cases (use a fixed test set to ensure that the experiments are fair across algorithms). The designed test cases may be grouped by increasing difficulty (e.g., simple cases without advanced tiles, cases with fragile tiles, cases with switches, and cases combining both features), and these groups should be clearly described in the report.

  - For each algorithm, measure:
    * Search time.
    * Memory usage (peak memory).
    * Number of expanded nodes.
    * Solution length (number of moves).
  - Present results with:
    * Tables summarizing the measurements.
    * Charts (plots) comparing the algorithms.
    * Your insights and discussion of the strengths and weaknesses of each algorithm.

- **References** and **Appendix**:
  - List all references (papers, books, online tutorials, GitHub repositories, etc.).
  - Include your Appendix, for example: how you used Generative AI (LLMs) in your work, what prompts you used.

3.7 Video (10 pts)

Upload a demo video to YouTube and provide the **public link** in your report. The video includes:

- **Graphical Interface (5 pts)**:
  - Demonstrate the GUI of your program.
  - Clearly show the detailed layout: the board with its different tile types (floor, void, goal, fragile, switches/bridges), the block in its various orientations, and the control buttons (e.g., New Game, Restart, Solve BFS/DFS/UCS/A*).
- **Feature Presentation (5 pts)**: Demonstrate the implemented functionalities:
  - Playing Bloxorz manually (rolling the block, triggering a switch, winning/losing a stage).
  - Running your solvers (e.g., choose a stage, run BFS/UCS/A*, show the found solution, possibly by animating or replaying the moves).

The video should contain subtitles or narration to make it easy to follow (either in English or Vietnamese).

4 Submission

Please follow the following submission guidelines:

- Your source code and report must be contributed in the form of a compressed file (.zip, .rar) and named according to the format `StudentID1_StudentID2_...`
- If the compressed file is larger than 20MB, upload it to Google Drive and share it via a link. **Absolutely no modifications are allowed after the deadline**.

Example details of the directory organization:

```
StudentID1_StudentID2_...
├── Source
│   ├── main.py
│   ├── README.md (how to run source code)
│   ├── requirements.txt (libraries to be installed)
│   └── ...
└── Report.pdf (included demo video URLs)
```

5 Notices

Please pay attention to the following notices:

- This is a **GROUP** assignment.
- Duration: about 3 weeks.
- AI tools are **not restricted**; however, students should use them wisely. Lab instructors have the right to conduct additional oral interviews with random groups to assess their knowledge of the project.
- Any form of plagiarism, dishonesty, or misconduct will result in a grade of zero for the course.

The end.